Assignment - 2

CHL-7201

Q1. Scalar RNN

$$h_t = \tanh(z_t), \quad z_t = W_{xh}x_t + W_{hh}h_{t-1} + b_h$$

$$y_t = W_{hy}h_t + b_y, \quad E_t = (d_t - y_t)^2$$

The training loss is only E = E2 + E3. Here h0 = 0.2.

(a) Forward pass

$$\tanh'(z_t) = 1 - h_t^2$$

t	z_t	h_t	tanh'	y_t	E_t
1	0.6200	0.5511	0.6963	0.6614	0.0683
2	0.2707	0.2643	0.9302	0.3171	0.0003
3	0.9786	0.7524	0.4338	0.9029	0.0106

$$z_1 = 0.8(0.5) + 0.6(0.2) + 0.1 = 0.6200$$

$$E = E_2 + E_3 = 0.0003 + 0.0106 = 0.0109$$

(b) Output weight

$$\frac{\partial E_t}{\partial y_t} = -2(d_t - y_t)$$

$$\frac{\partial E}{\partial W_{hy}} = \sum_{t=2}^{3} \frac{\partial E_t}{\partial y_t} h_t$$

$$\frac{\partial E_2}{\partial W_{hy}} = 0.0342 \times 0.2643 = 0.0090$$

$$\frac{\partial E_3}{\partial W_{hy}} = 0.2059 \times 0.7524 = 0.1549$$

$$\frac{\partial E}{\partial W_{hy}} = 0.0090 + 0.1549 = 0.1639$$

(c) BPTT for Whh and Wxh

As in the lecture, for each loss time T in {2, 3} and each earlier state i at most T,

$$\frac{\partial E_T}{\partial W} = \sum_{i=1}^{T} \frac{\partial E_T}{\partial h_T} \cdot \frac{\partial h_T}{\partial h_i} \cdot \frac{\partial h_i}{\partial W_{local}}$$

$$\frac{\partial E_T}{\partial h_T} = -2(d_T - y_T) W_{hy}$$

$$\frac{\partial h_j}{\partial h_{j-1}} = W_{hh} \tanh'(z_j)$$

$$\frac{\partial h_i}{\partial W_{hh\,local}} = \tanh'(z_i) h_{i-1}$$

$$\frac{\partial h_i}{\partial W_{xh\,local}} = \tanh'(z_i) x_i$$

Because h0 = 0.2 is not zero, the i = 1 term for Whh is kept. Both E2 and E3 are added.

$$\frac{\partial E_2}{\partial h_2} = 0.0411, \quad \frac{\partial E_3}{\partial h_3} = 0.2470$$

Backward chain:

$$\frac{\partial h_2}{\partial h_1} = 0.6 \times 0.9302 = 0.5581$$

$$\frac{\partial h_3}{\partial h_2} = 0.6 \times 0.4338 = 0.2603$$

$$\frac{\partial h_3}{\partial h_1} = 0.2603 \times 0.5581 = 0.1453$$

From E2. The chain column is dh2 / dhi.

i	chain	dh/dWhh	dh/dWxh	Whh	Wxh
1	0.5581	0.1393	0.3481	0.0032	0.0080
2	1	0.5126	-0.1860	0.0210	-0.0076

From E3. The chain column is dh3 / dhi.

i	chain	dh/dWhh	dh/dWxh	Whh	Wxh
1	0.1453	0.1393	0.3481	0.0050	0.0125
2	0.2603	0.5126	-0.1860	0.0330	-0.0120
3	1	0.1146	0.3905	0.0283	0.0965

$$\frac{\partial E}{\partial W_{hh}} = 0.0032 + 0.0210 + 0.0050 + 0.0330 + 0.0283 = 0.0905$$

The five Wxh entries above add to 0.0974 after each has been rounded. Before that rounding the sum is 0.0973, and that is the value that matches a finite-difference check. Use 0.0973 for the update. Either value gives the same new Wxh at four decimal places.

$$\frac{\partial E}{\partial W_{xh}} = 0.0973, \quad \frac{\partial E}{\partial W_{hh}} = 0.0905, \quad \frac{\partial E}{\partial W_{hy}} = 0.1639$$

(d) One gradient step, learning rate 0.5

Biases stay as they are. Only the three weights move.

$$W \leftarrow W - \eta \frac{\partial E}{\partial W}$$

$$W_{xh}: 0.8000 - 0.5(0.0973) = 0.7513$$
$$W_{hh}: 0.6000 - 0.5(0.0905) = 0.5547$$
$$W_{hy}: 1.2000 - 0.5(0.1639) = 1.1180$$

New forward pass: E2 = 0.0012, E3 = 0.0000, so

$$E_{new} = 0.0012$$

The loss fell, from 0.0109 to 0.0012.

(e) Vanishing gradients

(i) From the chain above,

$$\frac{\partial h_3}{\partial h_1} = 0.1453$$

(ii) From h1 to h50 there are 49 factors, and each satisfies $|W_{hh}| \tanh'$ at most $|W_{hh}| = 0.6$. So

$$\left|\frac{\partial h_{50}}{\partial h_1}\right| \le 0.6^{49} = 1.3471 \times 10^{-11}$$

(iii) On the GRU additive path the factor per step is 1 - z_t. With z_t = 0.1 for 49 steps,

$$(1 - 0.1)^{49} = 0.9^{49} = 0.0057$$

(iv) Clipping only shrinks a gradient whose norm is already above the threshold. The product

1.3×10^{-11}

sits far below any threshold, so the cap leaves it that small and the early-step update stays negligible. Truncated BPTT with chunk length k = 20 carries the hidden state forward through the whole sequence, but backpropagates only inside each chunk of 20 steps. A dependency that runs from step 1 to step 50 crosses a chunk boundary, so the loss at step 50 sends no gradient back to step 1. That long dependency is left untrained.

Q2. Encoder-decoder

$h_t = \tanh(W_x x_t + W_h h_{t-1} + b), \quad h_0 = 0, \quad c = h_3$

(a) Encoder

$z_1 = (0.5, -0.3), \quad h_1 = (0.4621, -0.2913)$

$h_2 = (0.3848, -0.2727)$

$h_3 = (-0.3569, 0.3286)$

$c = h_3 = (-0.3569, 0.3286)$

(b) Decoder step 1

$s_0 = c$, and the START embedding is (0, 0). Since $W_s = 0.5\,I$ and $W_c = I$,

$s_1 = \tanh(1.5\,c) = (-0.4895, 0.4565)$

Logits, in the order Normal, Alarm, END:

$(-0.3907, 1.4024, -0.4670)$

$e^{-0.3907} = 0.6766, \quad e^{1.4024} = 4.0649, \quad e^{-0.4670} = 0.6269$

The sum is 5.3684, so

$P = (0.1260, 0.7572, 0.1168)$

The predicted token is Alarm. The true token is Normal.

(c) Step 2, two ways

Teacher forcing, feed the true y_1^* = Normal, e = (1, 0):

$s_2 = (-0.9219, -0.4163)$

$P = (0.1991, 0.2571, 0.5438), \quad \hat{y}_2 = \text{END}$

Inference, feed the model's own y_1 = Alarm, e = (0, 1):

$s_2 = (-0.9219, 0.7844)$

$P = (0.0196, 0.9270, 0.0534), \quad \hat{y}_2 = \text{Alarm}$

During training the decoder is shown the true previous token, so step 2 is computed from Normal and it does emit END. At inference the wrong token Alarm is written back in, the second state is different, and the decoder emits Alarm again. If decoding stops at END or after 2 steps, inference produces (Alarm, Alarm). END has not appeared, so the run stops because of the 2-step limit.

Q3. Attention

The score of each encoder state is the dot product of the previous decoder state with that state. Softmax turns the scores into weights, and the context is their weighted sum. The decoder equation is the same as in Q2, with this new context.

$$e_{ti} = s_{t-1}^T h_i, \quad c_t = \sum_i \alpha_{ti} h_i$$

(a) Step 1, s0 = h3

$e_1 = (-0.2606, -0.2270, 0.2353)$

$\alpha_1 = (0.2720, 0.2813, 0.4467)$

$c_1 = (0.0745, -0.0092)$

$s_1 = (-0.1035, 0.1538)$

$P = (0.4393, 0.4056, 0.1551)$

y1 = Normal.

(b) Step 2, teacher forcing with Normal

$e_2 = (-0.0927, -0.0818, 0.0875)$

$\alpha_2 = (0.3117, 0.3151, 0.3732)$

$c_2 = (0.1321, -0.0541)$

$s_2 = (-0.7257, -0.7518)$

$P = (0.3377, 0.0976, 0.5648)$

y2 = END. The predicted sequence is (Normal, END), which is the true sequence.

(c) The two weight vectors

Step 1 puts the most weight on h3 (0.4467). Step 2 still puts the most weight on h3 (0.3732), with the three weights closer together. The scores depend on the previous decoder state. When that state changes,

the softmax changes, and the weighted sum of h1, h2 and h3 changes with it. That is why the context is rebuilt at every step.

(d) Mean pooling, each weight = 1/3

$c = (0.1633, -0.0785)$

$s_1 = (-0.0151, 0.0856)$

$P = (0.5302, 0.3198, 0.1500)$

(i) P(Normal) is 0.1260 when c = h3 (Q2), 0.4393 with attention, and 0.5302 with equal weights.

(ii) The values 1/3 are fixed. They do not depend on the previous decoder state, or on which encoder state matters. Training changes the encoder and decoder weights, so the hidden states change. The dot products then grow for the states that help the token being predicted, the weights move away from 1/3, and the pattern differs from one decoding step to the next.

Q4. VAE

$\mu = w_\mu^T x + b_\mu \quad \log\sigma^2 = w_v^T x + b_v$

$z = \mu + \sigma\varepsilon, \quad \hat{x} = \text{sigmoid}(W'z + b')$

$$L = \sum_{j=1}^{2} (x_j - \hat{x}_j)^2 + \frac{1}{2}(\mu^2 + \sigma^2 - \log\sigma^2 - 1)$$

(a) Encoder

$\mu = 0.5(0.8) + 0.5(0.4) = 0.6000$

$\log\sigma^2 = -1(0.8) + 0.5(0.4) - 0.6 = -1.2000$

$\sigma^2 = e^{-1.2} = 0.3012, \quad \sigma = e^{-0.6} = 0.5488$

$z = 0.6000 + 0.5488(0.5) = 0.8744$

(b) Decoder and loss

$u = (1.2488, -0.1256), \quad \hat{x} = (0.7771, 0.4686)$

$L_{rec} = (0.8 - 0.7771)^2 + (0.4 - 0.4686)^2 = 0.0052$

$L_{KL} = \frac{1}{2}(0.6000^2 + 0.3012 - (-1.2000) - 1) = 0.4306$

$L = 0.0052 + 0.4306 = 0.4358$

(c) Gradients through the reparameterisation

Let l stand for log of sigma squared, so sigma = e to the power l/2, and

$$\frac{\partial z}{\partial \ell} = \frac{\varepsilon \sigma}{2}$$

$$\frac{\partial \mathcal{L}_{rec}}{\partial \hat{x}_j} = -2(x_j - \hat{x}_j)$$

$$\frac{\partial \hat{x}_j}{\partial u_j} = \hat{x}_j(1 - \hat{x}_j)$$

$$\frac{\partial \mathcal{L}_{rec}}{\partial z} = 0.0183$$

The KL term contributes mu for the mean, and (sigma squared minus 1)/2 for l:

$$\frac{\partial \mathcal{L}}{\partial \mu} = 0.0183 + 0.6000 = 0.6183$$

$$\frac{\partial \mathcal{L}}{\partial \ell} = 0.0183 \cdot \frac{0.5 \times 0.5488}{2} + \frac{0.3012 - 1}{2} = -0.3469$$

Sampling z straight from N(mu, sigma^2) is a random node. It has no derivative, so the chain from L stops there and the encoder never receives a gradient. Writing z = mu + sigma epsilon, with epsilon drawn outside the graph, turns that step into addition and multiplication, which do have derivatives.

(d) Monitoring. Noise is 0, so z = mu

Normal sample x = (0.8, 0.4), and mu = 0.6000.

$$\hat{x} = (0.6682,\ 0.4013), \quad \text{error} = 0.0174$$

Suspect sample xf = (0.2, 0.9), and mu = 0.5500.

$$\hat{x} = (0.6457,\ 0.3894), \quad \text{error} = 0.4594$$

The suspect sample is the one to flag. Its reconstruction error is 0.4594 / 0.0174 = 26.44 times the normal sample's error.

(e) A new sample, z = -1

$$u = (-2.5,\ -2), \quad \hat{x} = (0.0759,\ 0.1192)$$

The KL term pulls q(z/x) toward N(0, 1), so the decoder is trained on codes spread over that prior. Drawing z = -1 from the prior and decoding is a draw from the learned model. A plain autoencoder stores one point per input and has no prior, so there is no distribution to draw z from.